claude-windows / bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f\subagents\workflows\wf_b8703ccc-78b\agent-a07317373cd85480c.jsonl`
- SHA-256: `941c5835597aa9b3411d9fb2e8bb5fa585242e8b01f15ee1025dcb5d3b9f7c25`
- Source modified: `2026-06-17T14:36:52+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8703ccc-78b`
- session_id: `bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f`

Transcript

1. user (2026-06-17T14:35:18.781Z)

Adversarially VERIFY this code-review finding against the ACTUAL code in undefined. Default to is_real=false unless you can point to the exact line that proves the bug is real and reachable. Finding: severity=minor; title="Error logging now shows stitch_exists and detect_exists separately"; where=tools/generate_highlights.py:133-134; why=Old: logger.warning('[%d/%d] SKIP %s ? source missing', ...). New: logger.warning(..., os.path.exists(src), os.path.exists(detect_src)). This provides more diagnostic info but changes the log output format. Not a functional regression, but log parsers may break.. Read the relevant file(s) and the tests. Is this a REAL, reachable problem in THIS diff (not a pre-existing issue, not a hypothetical)?

2. assistant (2026-06-17T14:35:20.405Z)

I'll verify this code-review finding by examining the actual code and tests. Let me start by reading the relevant file.

3. user (2026-06-17T14:35:20.650Z)

```
1       """Batch highlight-reel runner ? orchestrates "generate highlights for these videos
2       into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
3
4       Why this exists (lessons paid for the hard way):
5       - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work
dir
6         first, then COPIED into the destination (atomically: temp name -> os.replace). The
7         destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
8         Google-Drive-synced folder once lost finished reels + logs.)
9       - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
10        client (deleted files resurrect, partial writes sync). Generate locally; publish once.
11      - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
12        (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
13        duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
14      - **Resumable & concurrency-friendly.** A video whose reel already exists in the
15        destination is skipped (unless --force); launching a second request never undoes the
16        first's published reels. The Gemini path also isolates its chunk workdir per video
17        (PR for output/chunks/<video_id>) so concurrent videos don't collide.
18
19      Usage:
20        python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
21            --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
22        python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
23      """
24      from __future__ import annotations
25
26      import argparse
```

```
    import logging
    import os
    import shutil
    import sys
    import traceback
    from typing import Any, Dict, List, Optional, cast

    from dotenv import load_dotenv

    sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

    from backend.config import load_config
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.results import Failure
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("highlights_runner")

    # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
    _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
    _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")


    def _atomic_publish(local_path: str, dest_path: str) -> None:
        """Copy local_path -> dest_path so the destination never sees a partial file.

        Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
        anything else in the destination folder.
        """
        os.makedirs(os.path.dirname(dest_path), exist_ok=True)
        tmp = dest_path + ".publishing.tmp"
        shutil.copy2(local_path, tmp)
        os.replace(tmp, dest_path)


    def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                                  local_work_dir: str, db_path: str, force: bool = False,
                                  skip_ai_handoff: bool = False, wasb_stream: bool = False)
-> dict:
        """Generate one highlight reel per input video into out_dir. Returns a summary dict.

        Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.

        skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
handoff);
            candidate windows become the rallies. wasb_stream: have the WASB detector decode
the
            video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
#178).
        """
        os.makedirs(out_dir, exist_ok=True)
        os.makedirs(local_work_dir, exist_ok=True)
        needs_local = segmenter in _WSL_DETECTORS

        # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
        # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
        # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
        # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
        load_dotenv()

```

```
        cfg = load_config()
        # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get).
        # Logged by the segmenter itself once logging is configured below.
        if skip_ai_handoff:
            cfg.indexing.skip_ai_handoff = True
        if wasb_stream:
            cfg.indexing.wasb.stream_video = True
        db = Database(db_path)
        seg_cls = segmenter_registry.get(segmenter)
        if seg_cls is None:
            raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")

        # Combined batch log (local; published at the end).
        fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
        if not logging.getLogger().handlers:
            logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
            for h in logging.getLogger().handlers:
                h.setFormatter(fmt)
        batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
        batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
        batch_fh.setFormatter(fmt)
        logging.getLogger().addHandler(batch_fh)

        summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
        logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
                    len(video_paths), out_dir, segmenter, force)

        for i, src in enumerate(video_paths, 1):
            name = os.path.basename(src)
            stem = os.path.splitext(name)[0]
            reel_name = f"{stem} - Highlights.mp4"
            dest_reel = os.path.join(out_dir, reel_name)
            local_reel = os.path.join(local_work_dir, reel_name)

            if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
                logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
                summary["skipped"].append(name)
                continue
            if not os.path.exists(src):
                logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
                summary["failed"].append(name)
                continue

            run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
            run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
            run_fh.setFormatter(fmt)
            logging.getLogger().addHandler(run_fh)
            staged_src: Optional[str] = None
            try:
                logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
                if needs_local:
                    staged_src = os.path.join(local_work_dir, name)
                    if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
                        logger.info("localizing source (%.0f MB)...", os.path.getsize(src) /
1024 / 1024)
                        shutil.copy2(src, staged_src)
                    proc_src = staged_src
```

```
                else:
                    proc_src = src

                vid = compute_video_id(proc_src)
                db.clear_video_data(vid)          # idempotent: no cross-run accumulation
                db.add_video(vid, proc_src, "processed", [])
                logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

                # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
                # main.py); cast keeps the type checker honest about that contract.
                result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
                if isinstance(result, Failure):
                    logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
                    summary["failed"].append(name)
                    continue

                segs = db.query_play_segments(vid, {})
                pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
                logger.info("rallies detected: %d", len(pairs))
                if not pairs:
                    logger.warning("no rallies -> no reel")
                    summary["no_rallies"].append(name)
                    continue

                # Compile locally, then publish atomically into the destination.
                VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
                _atomic_publish(local_reel, dest_reel)
                try:
                    os.remove(local_reel)
                except OSError:
                    pass
                logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
                summary["done"].append(name)
            except Exception:
                logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
                summary["failed"].append(name)
            finally:
                logging.getLogger().removeHandler(run_fh)
                run_fh.close()
                # publish this video's run.log (additive; never deletes anything in out_dir)
                try:
                    _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
                except OSError:
                    pass
                if staged_src and os.path.exists(staged_src):
                    try:
                        os.remove(staged_src)
                    except OSError:
                        pass

        logging.getLogger().removeHandler(batch_fh)
        batch_fh.close()
        try:
            _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
        except OSError:
            pass
        logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
        return summary


    def _resolve_videos(args) -> List[str]:
        if args.video:
            return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
```

```
in args.video]
204         if args.videos_dir:
205             return sorted(
206                 os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
207                 if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
208             )
209         raise SystemExit("provide --videos-dir or one/more --video")
210
211
212     def main() -> None:
213         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
214         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
215         ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
216         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
217         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
218         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
219                         help="local staging dir (reels compiled here, then published)")
220         ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
221         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
222         ap.add_argument("--skip-ai-handoff", action="store_true",
223                         help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
224                              "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
225         ap.add_argument("--wasb-stream", action="store_true",
226                         help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
227                              "dumping ~46k PNGs. Only affects wasb_hybrid.")
228         args = ap.parse_args()
229         generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
230                                   local_work_dir=args.local_work, db_path=args.db,
force=args.force,
231                                   skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
232
233
234     if __name__ == "__main__":
235         main()
236
```

4. assistant (2026-06-17T14:35:22.605Z)

Now let me check lines 133-134 more carefully and look for any recent changes. The code I see shows line 124 has the original warning message. Let me search for any git diff to see what changed:

5. user (2026-06-17T14:35:23.321Z)

(Bash completed with no output)

6. assistant (2026-06-17T14:35:24.464Z)

Let me check the current branch and recent commits:

7. user (2026-06-17T14:35:24.978Z)

f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)

```
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
(#200)
7e8f88f feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU) (#199)
cf8973e docs(eval): R2 design ? task-faithful tolerance-match rally metric (FOR REVIEW) (#198)
369e90c docs(eval): golden-labeled video registry (the growing golden-finalized set) (#197)
c483346 docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-
set discipline (#196)
bb4d6e2 feat(eval): Tier 1 (W1.5) ? hard-cap fallback enforces the window cap on continuously-
active trajectories (#195)
7d84319 fix(eval): golden-fixtures audit hardening ? close source_note/label leak + 16 confirmed
audit fixes (#194)
```

8. assistant (2026-06-17T14:35:26.319Z)

Let me check if there's a staged diff or if we're in the middle of a PR:

9. user (2026-06-17T14:35:26.556Z)

```
On branch docs/golden-regression-fixtures-plan
Your branch and 'origin/docs/golden-regression-fixtures-plan' have diverged,
and have 18 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        artifacts/
        docs/KHELSUTRA_PRODUCT_OVERVIEW.md

nothing added to commit but untracked files present (use "git add" to track)
```

10. assistant (2026-06-17T14:35:27.828Z)

Let me check if there's a PR branch or if we're looking at uncommitted changes. Let me see the diff to the base branch:

11. user (2026-06-17T14:35:28.195Z)

```
fatal: bad revision 'origin/main'
```

12. assistant (2026-06-17T14:35:34.498Z)

Let me check what branch exists:

13. user (2026-06-17T14:35:34.852Z)

```
+ bhi-execpolicy
* docs/golden-regression-fixtures-plan
+ docs/golden-videos-tracker
+ docs/quality-iterations-realfootage
+ docs/rally-quality-research
  feat/highlights-runner
+ feat/local-no-ai-segmenter
  feat/r1-milestone-config-flip
  feat/r1-net-overseg-guard
+ feat/r2-tolerance-metric
+ feat/r4-inactivity-end
  feat/r5-blob-splitter
+ feat/tier0-rally-seg-eval
+ feat/w1-hard-cap-fallback
+ feat/w2-netcross-gate
+ feat/w2-never-empty-safeguard
  fix/compiler-dedup-overlapping-segments
```

```
  fix/gemini-per-video-chunk-dir
+ fix/runner-stitch-watermark
  fix/wasb-test-mocks-readability-preflight
  fix/wsl-unreadable-source-preflight
+ master
+ perf/wasb-fast-frames
+ run/boxhill17-highlights
  worktree-agent-a771ff1d85e18f2ed
  remotes/origin/HEAD -> origin/master
  remotes/origin/docs/golden-regression-fixtures-plan
  remotes/origin/docs/golden-videos-tracker
  remotes/origin/docs/pmf-field-signals
  remotes/origin/docs/quality-iterations-realfootage
  remotes/origin/docs/r2-eval-metric
  remotes/origin/docs/rally-quality-research
  remotes/origin/feat/golden-fixtures-audit-hardening
  remotes/origin/feat/golden-fixtures-crossos
  remotes/origin/feat/golden-fixtures-doc
  remotes/origin/feat/golden-fixtures-m1
  remotes/origin/feat/golden-fixtures-p1
  remotes/origin/feat/highlights-runner
  remotes/origin/feat/local-no-ai-segmenter
  remotes/origin/feat/nightly-regression
  remotes/origin/feat/r1-milestone-config-flip
  remotes/origin/feat/r1-net-overseg-guard
  remotes/origin/feat/r2-tolerance-metric
  remotes/origin/feat/r4-inactivity-end
  remotes/origin/feat/r5-blob-splitter
  remotes/origin/feat/tier0-rally-seg-eval
  remotes/origin/feat/tier1-bounded-windowing
  remotes/origin/feat/w1-hard-cap-fallback
  remotes/origin/feat/w2-netcross-gate
  remotes/origin/feat/w2-never-empty-safeguard
  remotes/origin/fix/compiler-dedup-overlapping-segments
  remotes/origin/fix/gemini-per-video-chunk-dir
  remotes/origin/fix/runner-stitch-watermark
  remotes/origin/fix/wasb-test-mocks-readability-preflight
  remotes/origin/fix/wsl-unreadable-source-preflight
  remotes/origin/master
  remotes/origin/perf/wasb-fast-frames
```

14. assistant (2026-06-17T14:35:36.764Z)

The finding says this is in "undefined" and references lines 133-134 with `stitch_exists` and `detect_exists` variables. Let me search for these strings in the file to see if they actually exist:

15. user (2026-06-17T14:35:37.421Z)

No files found

16. assistant (2026-06-17T14:35:39.668Z)

This confirms that `stitch_exists` and `detect_exists` do NOT exist in the current file. Let me also check if there's any reference to these in the working tree at all:

17. user (2026-06-17T14:35:40.162Z)

No files found

18. assistant (2026-06-17T14:35:41.900Z)

Let me check the actual line 124 to confirm the current state:

19. user (2026-06-17T14:35:42.120Z)

```
120                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
121                 summary["skipped"].append(name)
122                 continue
123             if not os.path.exists(src):
124                 logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
125                 summary["failed"].append(name)
126                 continue
127
128             run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
129             run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
```

20. assistant (2026-06-17T14:35:44.365Z)

Perfect. The current code at line 124 shows:
```python
logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
```

This is the original format with 3 parameters. The finding claims the NEW code should have:
```python
logger.warning(..., os.path.exists(src), os.path.exists(detect_src))
```

Let me search for any branch that might have this change or look at git log for recent commits to this file:

21. user (2026-06-17T14:35:45.048Z)

```
a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
b6a39c4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)
3ea7ad4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load)
4942d51 feat(tools): batch highlight RUNNER ? clean, non-destructive, Drive-safe reel generation
(#182)
bd3c8b5 feat(tools): batch highlight RUNNER ? clean, non-destructive, Drive-safe reel generation
```

22. assistant (2026-06-17T14:35:47.947Z)

Let me check that fix commit which mentions "detect-from":

23. user (2026-06-17T14:35:48.436Z)

```
commit a6adcc57290bd65c3ca7c092d0568ff02cf60378
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Wed Jun 17 19:59:43 2026 +0530

    fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels

    The batch runner (tools/generate_highlights.py) constructed VideoCompiler bare ?
    `VideoCompiler(local_work_dir).compile_highlights(...)` ? so reels shipped UN-watermarked
    (watermark defaults to None/off) and used the ctor's begin/end padding (0.5/1.0) instead of
    config.json `stitching.{begin_padding,end_padding}` (0.5/0.5). Every other compile call site
    threads stitching config; this one didn't. Now it reads `stitching.*` dict-compatibly and
    passes watermark + paddings through (the on-by-default branding mark).

    Also add `--detect-from`: run the detector on a cheaper proxy (e.g. a 1080p downscale of a
4K
    source) while stitching the reel from the full-res --video. Timestamps transfer 1:1 (pure
    temporal-preserving downscale), so it's the bandwidth/GPU-saver split the workflow wanted ?
    detect on the proxy, deliver a 4K reel. Reel is named after the --video (the thing watched).
```

```
    Tests: extend test_generate_highlights.py ? watermark+padding are threaded into
VideoCompiler
    (regression lock), --detect-from keys detection off the proxy but stitches from the
original,
    and _resolve_detect_from validates 1:1 pairing.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
index ae9f045..565663a 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -63,7 +63,8 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:

 def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                               local_work_dir: str, db_path: str, force: bool = False,
-                              skip_ai_handoff: bool = False, wasb_stream: bool = False) ->
dict:
+                              skip_ai_handoff: bool = False, wasb_stream: bool = False,
+                              detect_from_paths: Optional[List[str]] = None) -> dict:
     """Generate one highlight reel per input video into out_dir. Returns a summary dict.

     Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
@@ -71,6 +72,11 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
     skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
         candidate windows become the rallies. wasb_stream: have the WASB detector decode the
         video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
+    detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
+        detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
+        reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
+        (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
+        same source (legacy behaviour).
     """
     os.makedirs(out_dir, exist_ok=True)
     os.makedirs(local_work_dir, exist_ok=True)
@@ -115,13 +121,17 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
         reel_name = f"{stem} - Highlights.mp4"
         dest_reel = os.path.join(out_dir, reel_name)
         local_reel = os.path.join(local_work_dir, reel_name)
+        # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
+        # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
+        detect_src = detect_from_paths[i - 1] if detect_from_paths else src

         if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
             logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
             summary["skipped"].append(name)
             continue
-        if not os.path.exists(src):
-            logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
+        if not os.path.exists(src) or not os.path.exists(detect_src):
+            logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
+                           i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
             summary["failed"].append(name)
             continue

@@ -132,23 +142,29 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
         staged_src: Optional[str] = None
         try:
             logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
+            # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
```

```
+            # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile,
so
+            # only the detector input ever needs staging.
             if needs_local:
-                staged_src = os.path.join(local_work_dir, name)
-                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
-                    logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
-                    shutil.copy2(src, staged_src)
-                proc_src = staged_src
+                staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
+                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
+                    logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
+                    shutil.copy2(detect_src, staged_src)
+                proc_detect = staged_src
             else:
-                proc_src = src
+                proc_detect = detect_src

-            vid = compute_video_id(proc_src)
+            vid = compute_video_id(proc_detect)
             db.clear_video_data(vid)          # idempotent: no cross-run accumulation
-            db.add_video(vid, proc_src, "processed", [])
+            db.add_video(vid, proc_detect, "processed", [])
+            if detect_src != src:
+                logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
+                            os.path.basename(detect_src), name)
             logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

             # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
             # main.py); cast keeps the type checker honest about that contract.
-            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
+            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
             if isinstance(result, Failure):
                 logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                 summary["failed"].append(name)
@@ -162,8 +178,20 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
                 summary["no_rallies"].append(name)
                 continue

-            # Compile locally, then publish atomically into the destination.
-            VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+            # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
+            # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
+            stitch_src = src if detect_from_paths else proc_detect
+            # Honor the configured branding watermark + segment paddings (stitching.*). Read
+            # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
+            # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
+            # ctor default paddings; this threads config.json through (the on-by-default mark).
+            st = cfg.get("stitching") or {}
+            VideoCompiler(
+                local_work_dir,
+                begin_padding=float(st.get("begin_padding", 0.5)),
+                end_padding=float(st.get("end_padding", 0.5)),
+                watermark=st.get("watermark"),
+            ).compile_highlights(stitch_src, pairs, reel_name)
             _atomic_publish(local_reel, dest_reel)
             try:
                 os.remove(local_reel)
```

```
@@ -209,6 +237,21 @@ def _resolve_videos(args) -> List[str]:
     raise SystemExit("provide --videos-dir or one/more --video")


+def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
+    """Resolve --detect-from into a list parallel to video_paths (or None if unused).
+
+    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
+    proxy maps to exactly one --video by position."""
+    if not getattr(args, "detect_from", None):
+        return None
+    if not args.video:
+        raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
+    if len(args.detect_from) != len(args.video):
+        raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
+                         f"--video count ({len(args.video)}); pair them 1:1 in order")
+    return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
+
+
 def main() -> None:
     ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
     ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
--video given)")
@@ -225,10 +268,18 @@ def main() -> None:
     ap.add_argument("--wasb-stream", action="store_true",
                     help="WASB decode-on-the-fly fast path (#178; output-preserving), instead
of "
                          "dumping ~46k PNGs. Only affects wasb_hybrid.")
+    ap.add_argument("--detect-from", action="append",
+                    help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
+                         "(cheaper to decode) while the reel is stitched from the matching
--video "
+                         "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
--video "
+                         "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
     args = ap.parse_args()
-    generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
+    video_paths = _resolve_videos(args)
+    detect_from_paths = _resolve_detect_from(args, video_paths)
+    generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                               local_work_dir=args.local_work, db_path=args.db,
force=args.force,
-                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
+                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
+                              detect_from_paths=detect_from_paths)


 if __name__ == "__main__":
```

24. assistant (2026-06-17T14:35:51.407Z)

Perfect! Now I can see the actual change. Looking at the diff for lines 131-134 (in the new
version):

```python
        if not os.path.exists(src) or not os.path.exists(detect_src):
            logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
                           i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
```

```

Let me check if this commit is currently in the working tree:

**25. user (2026-06-17T14:35:52.227Z)**

(Bash completed with no output)

**26. assistant (2026-06-17T14:35:53.708Z)**

The commit is not in the current branch. Let me verify by checking if the current code has the new change:

**27. user (2026-06-17T14:35:54.112Z)**

```
123   if not os.path.exists(src):
124   logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
125   summary["failed"].append(name)
126   continue
127
```

**28. assistant (2026-06-17T14:35:57.042Z)**

The current code does NOT have the change described in the finding. Now let me look at the test file to see if it covers this scenario:

**29. user (2026-06-17T14:35:58.027Z)**

```
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-
a771ff1d85e18f2ed/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/boxhill17/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-
golden/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-rally-
quality/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-
today/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/hardchop/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hardchop/tests/__pycache__
/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/local-
noai/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/local-
noai/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-
stitch/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-
stitch/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/r2-impl/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/r2-
impl/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/r4-endrule/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/r4-
endrule/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/tier0-eval/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier0-
```
```

```
eval/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/tier1-windowing/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-
windowing/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/w2-netcross/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-
netcross/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/w2-safeguard/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-
safeguard/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/badminton-highlight-indexer/tests/test_generate_highlights.py
/c/Users/avidu/Projects/badminton-highlight-
indexer/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
```

30. assistant (2026-06-17T14:35:59.218Z)

Let me check the main project test file:

31. user (2026-06-17T14:35:59.563Z)

```
1       """tools/generate_highlights.py ? the batch highlight runner. The load-bearing
guarantees
2       are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
3       (heavy deps ? segmenter/compiler/db ? are mocked)."""
4       import os
5       from unittest.mock import MagicMock, patch
6
7       import tools.generate_highlights as gh
8
9
10      def test_atomic_publish_is_nondestructive(tmp_path):
11          src = tmp_path / "local_reel.mp4"
12          src.write_bytes(b"reel-bytes")
13          dest_dir = tmp_path / "out"
14          dest_dir.mkdir()
15          sibling = dest_dir / "Someone Else - Highlights.mp4"
16          sibling.write_bytes(b"do-not-touch")
17
18          gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))
19
20          assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
21          assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
22          assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind
23
24
25      def test_resolve_videos_explicit_and_dir(tmp_path):
26          (tmp_path / "a.MP4").write_bytes(b"x")
27          (tmp_path / "b.mp4").write_bytes(b"x")
28          (tmp_path / "notes.txt").write_text("ignore")
29          args = MagicMock(video=None, videos_dir=str(tmp_path))
30          found = gh._resolve_videos(args)
31          assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only,
sorted
32          args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
33          assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]
34
35
36      def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
37          """force=False: a video whose reel already exists is skipped, the segmenter is never
38          called, and unrelated files already in the destination are left intact."""
39          out = tmp_path / "out"
40          out.mkdir()
```

```
41          (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
42          (out / "Adarsh v Avi - run.log").write_text("prior log")
43          other = out / "Someone Else - Highlights.mp4"
44          other.write_bytes(b"untouched")
45          src = tmp_path / "Adarsh v Avi.MP4"
46          src.write_bytes(b"src")
47
48          seg_registry = MagicMock()
49          with patch.object(gh, "load_config", return_value={}), \
50               patch.object(gh, "Database", return_value=MagicMock()), \
51               patch.object(gh, "segmenter_registry", seg_registry):
52              summary = gh.generate_highlights_batch(
53                  [str(src)], str(out), segmenter="gemini",
54                  local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)
55
56          assert summary["skipped"] == ["Adarsh v Avi.MP4"]
57          # never processed: the resolved segmenter class was never instantiated/run
58          seg_registry.get.return_value.assert_not_called()
59          assert other.read_bytes() == b"untouched"            # other work preserved
60          assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"
# not regenerated
61
62
63      def test_batch_applies_local_overrides(tmp_path):
64          """--skip-ai-handoff / --wasb-stream inject into the typed config before
segmentation."""
65          from backend.config import AppConfig
66          cfg = AppConfig()
67          assert cfg.indexing.skip_ai_handoff is False          # default off
68          assert cfg.indexing.wasb.stream_video is False
69
70          out = tmp_path / "out"
71          out.mkdir()
72          src = tmp_path / "clip.MP4"
73          src.write_bytes(b"src")
74
75          seg_instance = MagicMock()
76          seg_instance.process_video.return_value = []          # no rallies -> no
compile/publish
77          seg_registry = MagicMock()
78          seg_registry.get.return_value = MagicMock(return_value=seg_instance)
79
80          with patch.object(gh, "load_config", return_value=cfg), \
81               patch.object(gh, "Database", return_value=MagicMock()), \
82               patch.object(gh, "compute_video_id", return_value="vid123"), \
83               patch.object(gh, "segmenter_registry", seg_registry):
84              gh.generate_highlights_batch(
85                  [str(src)], str(out), segmenter="gemini",
86                  local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
87                  force=True, skip_ai_handoff=True, wasb_stream=True)
88
89          assert cfg.indexing.skip_ai_handoff is True
90          assert cfg.indexing.wasb.stream_video is True
91
```

32. assistant (2026-06-17T14:36:01.556Z)

Now let me check if commit a6adcc5 is reachable in any branch or if it's on a separate branch.
Let me look at the test file in the pr-stitch worktree since we're in that directory:

33. user (2026-06-17T14:36:02.154Z)

```
1       """tools/generate_highlights.py ? the batch highlight runner. The load-bearing
guarantees
```

```
    are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
    (heavy deps ? segmenter/compiler/db ? are mocked)."""
    import os
    from unittest.mock import MagicMock, patch

    import pytest

    import tools.generate_highlights as gh


    def _fake_compiler(captured):
        """A stand-in VideoCompiler that records its ctor kwargs + compile args and writes
the
        expected reel file (so the runner's atomic-publish step succeeds)."""
        class FakeCompiler:
            def __init__(self, output_dir, end_padding=1.0, begin_padding=0.5,
watermark=None):
                captured["ctor"] = dict(output_dir=output_dir, end_padding=end_padding,
                                        begin_padding=begin_padding, watermark=watermark)

            def compile_highlights(self, raw_video_path, segments, output_filename):
                captured["compile"] = dict(raw=raw_video_path, segments=list(segments),
                                           out=output_filename)
                reel = os.path.join(captured["ctor"]["output_dir"], output_filename)
                with open(reel, "wb") as fh:
                    fh.write(b"reel-bytes")
                return reel

        return FakeCompiler


    def test_atomic_publish_is_nondestructive(tmp_path):
        src = tmp_path / "local_reel.mp4"
        src.write_bytes(b"reel-bytes")
        dest_dir = tmp_path / "out"
        dest_dir.mkdir()
        sibling = dest_dir / "Someone Else - Highlights.mp4"
        sibling.write_bytes(b"do-not-touch")

        gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))

        assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
        assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
        assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind


    def test_resolve_videos_explicit_and_dir(tmp_path):
        (tmp_path / "a.MP4").write_bytes(b"x")
        (tmp_path / "b.mp4").write_bytes(b"x")
        (tmp_path / "notes.txt").write_text("ignore")
        args = MagicMock(video=None, videos_dir=str(tmp_path))
        found = gh._resolve_videos(args)
        assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only,
sorted
        args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
        assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]


    def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
        """force=False: a video whose reel already exists is skipped, the segmenter is never
        called, and unrelated files already in the destination are left intact."""
        out = tmp_path / "out"
        out.mkdir()
        (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
        (out / "Adarsh v Avi - run.log").write_text("prior log")
```

```
        other = out / "Someone Else - Highlights.mp4"
        other.write_bytes(b"untouched")
        src = tmp_path / "Adarsh v Avi.MP4"
        src.write_bytes(b"src")

        seg_registry = MagicMock()
        with patch.object(gh, "load_config", return_value={}), \
             patch.object(gh, "Database", return_value=MagicMock()), \
             patch.object(gh, "segmenter_registry", seg_registry):
            summary = gh.generate_highlights_batch(
                [str(src)], str(out), segmenter="gemini",
                local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)

        assert summary["skipped"] == ["Adarsh v Avi.MP4"]
        # never processed: the resolved segmenter class was never instantiated/run
        seg_registry.get.return_value.assert_not_called()
        assert other.read_bytes() == b"untouched"            # other work preserved
        assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"
# not regenerated


    def test_batch_applies_local_overrides(tmp_path):
        """--skip-ai-handoff / --wasb-stream inject into the typed config before
segmentation."""
        from backend.config import AppConfig
        cfg = AppConfig()
        assert cfg.indexing.skip_ai_handoff is False          # default off
        assert cfg.indexing.wasb.stream_video is False

        out = tmp_path / "out"
        out.mkdir()
        src = tmp_path / "clip.MP4"
        src.write_bytes(b"src")

        seg_instance = MagicMock()
        seg_instance.process_video.return_value = []          # no rallies -> no
compile/publish
        seg_registry = MagicMock()
        seg_registry.get.return_value = MagicMock(return_value=seg_instance)

        with patch.object(gh, "load_config", return_value=cfg), \
             patch.object(gh, "Database", return_value=MagicMock()), \
             patch.object(gh, "compute_video_id", return_value="vid123"), \
             patch.object(gh, "segmenter_registry", seg_registry):
            gh.generate_highlights_batch(
                [str(src)], str(out), segmenter="gemini",
                local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
                force=True, skip_ai_handoff=True, wasb_stream=True)

        assert cfg.indexing.skip_ai_handoff is True
        assert cfg.indexing.wasb.stream_video is True


    def _run_one(tmp_path, *, video_paths, detect_from_paths=None, segmenter="gemini"):
        """Drive generate_highlights_batch once with a fake compiler/segmenter/db, returning
the
        captured VideoCompiler ctor+compile args and the compute_video_id mock."""
        from backend.config import AppConfig
        cfg = AppConfig()                                     # watermark on by default
        out = tmp_path / "out"
        out.mkdir()
        db = MagicMock()
        db.query_play_segments.return_value = [{"start_time": 1.0, "end_time": 5.0}]
        seg_instance = MagicMock()
```

```
        seg_instance.process_video.return_value = ["ok"]    # non-Failure
        seg_registry = MagicMock()
        seg_registry.get.return_value = MagicMock(return_value=seg_instance)
        captured: dict = {}
        cvid = MagicMock(return_value="vid123")
        with patch.object(gh, "load_config", return_value=cfg), \
             patch.object(gh, "Database", return_value=db), \
             patch.object(gh, "compute_video_id", cvid), \
             patch.object(gh, "VideoCompiler", _fake_compiler(captured)), \
             patch.object(gh, "segmenter_registry", seg_registry):
            gh.generate_highlights_batch(
                [str(p) for p in video_paths], str(out), segmenter=segmenter,
                local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
                force=True, detect_from_paths=[str(p) for p in detect_from_paths] if
detect_from_paths else None)
        return captured, cvid, out


    def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
        """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to
VideoCompiler
        (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
        src = tmp_path / "clip.MP4"
        src.write_bytes(b"src")
        captured, _cvid, out = _run_one(tmp_path, video_paths=[src])

        wm = captured["ctor"]["watermark"]
        assert wm and wm.get("enabled") is True                 # on-by-default mark
threaded through
        assert wm.get("text") == "Generated by Khelsutra.guru"
        assert captured["ctor"]["begin_padding"] == 0.5         # config paddings, not the
ctor 1.0 default
        assert captured["ctor"]["end_padding"] == 0.5
        assert captured["compile"]["raw"] == str(src)           # stitches from the (only)
source
        assert (out / "clip - Highlights.mp4").exists()         # published


    def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
        """--detect-from: the detector runs on the proxy (video_id keyed off it) but the
reel is
        stitched from the full-res --video, and named after it."""
        orig = tmp_path / "GX01.MP4"
        orig.write_bytes(b"original-4k")
        proxy = tmp_path / "GX01_1080p.mp4"
        proxy.write_bytes(b"proxy-1080")
        captured, cvid, out = _run_one(tmp_path, video_paths=[orig],
detect_from_paths=[proxy])

        cvid.assert_called_once_with(str(proxy))                # detection keyed off the
proxy
        assert captured["compile"]["raw"] == str(orig)          # reel stitched from the
original
        assert (out / "GX01 - Highlights.mp4").exists()         # named after the original,
not the proxy


    def test_resolve_detect_from_validates_pairing(tmp_path):
        assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
        with pytest.raises(SystemExit):                         # count mismatch
            gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
        with pytest.raises(SystemExit):                         # requires explicit --video
            gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None,
videos_dir="d"), ["v1"])
```

```
177         abs = os.path.join(os.sep, "abs", "p.mp4")
178         assert gh._resolve_detect_from(
179             MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
180
```

34. assistant (2026-06-17T14:36:05.056Z)

Interesting! The pr-stitch worktree test file has the newer tests (including `test_resolve_detect_from_validates_pairing` and `test_batch_detect_from_proxy_stitches_from_original`) which test the new `detect_from` feature. This means commit a6adcc5 IS in this worktree. Let me check the actual code file in pr-stitch:

35. user (2026-06-17T14:36:05.895Z)

```
125             # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
126             detect_src = detect_from_paths[i - 1] if detect_from_paths else src
127
128             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
129                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
130                 summary["skipped"].append(name)
131                 continue
132             if not os.path.exists(src) or not os.path.exists(detect_src):
133                 logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
134                                i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
```

36. assistant (2026-06-17T14:36:08.746Z)

Perfect! Now I have the actual code with the change. Lines 132-134 in the pr-stitch worktree show:

```python
        if not os.path.exists(src) or not os.path.exists(detect_src):
            logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
                           i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
```

This is the ACTUAL code. Now let me verify:

1. Is the condition reachable?
2. Does the log format change break any log parsers?

Let me check if there are any log parsers or format-dependent code:

37. user (2026-06-17T14:36:09.008Z)

Found 1 file
C:\Users\avidu\Projects\badminton-highlight-indexer\tools\generate_highlights.py

38. assistant (2026-06-17T14:36:10.414Z)

Let me check for any log parsing or assertions about the log format in tests:

39. user (2026-06-17T14:36:10.816Z)

(Bash completed with no output)

40. assistant (2026-06-17T14:36:12.233Z)

Good - there are no tests that assert on the log format. Now let me check if there's any code

that parses this warning message (outside tests):

41. user (2026-06-17T14:36:18.706Z)

```
./.claude/worktrees/agent-a771ff1d85e18f2ed/tools/generate_highlights.py:
logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/boxhill17/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/docs-golden/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/docs-rally-quality/tools/generate_highlights.py:
logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/docs-today/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/hardchop/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/local-noai/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/pr-stitch/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing (stitch_exists=%s detect_exists=%s)",
./.claude/worktrees/r2-impl/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/r4-endrule/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/tier0-eval/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/tier1-windowing/tools/generate_highlights.py:
logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/w2-netcross/tools/generate_highlights.py:            logger.warning("[%d/%d]
SKIP %s ? source missing", i, len(video_paths), name)
./.claude/worktrees/w2-safeguard/tools/generate_highlights.py:
logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
./scratch/make_boxhill_highlights.py:        log.info("[%d/%d] SKIP %s ? source missing", i,
len(VIDEOS), name)
./tools/generate_highlights.py:            logger.warning("[%d/%d] SKIP %s ? source missing", i,
len(video_paths), name)
```

42. assistant (2026-06-17T14:36:20.797Z)

Interesting - only the pr-stitch worktree has the new format. Let me check if there are any log parsers in the codebase:

43. user (2026-06-17T14:36:22.569Z)

```
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/automated_provider.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/loader.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/ablation.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/annotations.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/experiment.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gt_loader.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/harness.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/regression.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/database.py
xargs: grep: terminated by signal 13
```

44. assistant (2026-06-17T14:36:23.826Z)

Let me check for any tests that might validate the log output format specifically:

45. user (2026-06-17T14:36:24.472Z)

(Bash completed with no output)

46. assistant (2026-06-17T14:36:26.391Z)

Let me check what the test coverage looks like for the error condition:

47. user (2026-06-17T14:36:27.039Z)

(Bash completed with no output)

48. assistant (2026-06-17T14:36:32.285Z)

So there's no test for the "source missing" case in pr-stitch. Let me check if the code path is actually reachable by understanding the condition:

The new code has:
```python
if not os.path.exists(src) or not os.path.exists(detect_src):
    logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s detect_exists=%s)",
                   i, len(video_paths), name, os.path.exists(src), os.path.exists(detect_src))
```

This is reachable IF:
1. `src` doesn't exist, OR
2. `detect_src` doesn't exist

Both are possible conditions. Now let me verify the actual log output format more carefully. The finding claims this changes from:
- OLD: `logger.warning('[%d/%d] SKIP %s ? source missing', ...)` (3 format args)
- NEW: `logger.warning(..., os.path.exists(src), os.path.exists(detect_src))` (5 format args total)

The new version in pr-stitch shows:
```python
logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s detect_exists=%s)",
               i, len(video_paths), name, os.path.exists(src), os.path.exists(detect_src))
```

This is a format string with 5 `%s/%%d/%%s` placeholders... wait, let me count again:
- `[%d/%d]` = 2 placeholders for `i` and `len(video_paths)`
- `%s` = 1 placeholder for `name`
- `%s` (in the parentheses) = 1 placeholder for `os.path.exists(src)` (a boolean)
- `%s` = 1 placeholder for `os.path.exists(detect_src)` (a boolean)

Total = 5 format specifiers, 5 arguments. The format is correct.

Now the critical question: **Is this a REAL problem in THIS diff?**

Let me verify by looking at tests to see if they're checking log format anywhere:

49. user (2026-06-17T14:36:32.681Z)

```
140
141     def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
142         """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to
VideoCompiler
143         (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
144         src = tmp_path / "clip.MP4"
145         src.write_bytes(b"src")
146         captured, _cvid, out = _run_one(tmp_path, video_paths=[src])
147
148         wm = captured["ctor"]["watermark"]
149         assert wm and wm.get("enabled") is True                 # on-by-default mark
threaded through
150         assert wm.get("text") == "Generated by Khelsutra.guru"
151         assert captured["ctor"]["begin_padding"] == 0.5         # config paddings, not the
```

```
ctor 1.0 default
152         assert captured["ctor"]["end_padding"] == 0.5
153         assert captured["compile"]["raw"] == str(src)           # stitches from the (only)
source
154         assert (out / "clip - Highlights.mp4").exists()         # published
155
156
157     def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
158         """--detect-from: the detector runs on the proxy (video_id keyed off it) but the
reel is
159         stitched from the full-res --video, and named after it."""
160         orig = tmp_path / "GX01.MP4"
161         orig.write_bytes(b"original-4k")
162         proxy = tmp_path / "GX01_1080p.mp4"
163         proxy.write_bytes(b"proxy-1080")
164         captured, cvid, out = _run_one(tmp_path, video_paths=[orig],
detect_from_paths=[proxy])
165
166         cvid.assert_called_once_with(str(proxy))                # detection keyed off the
proxy
167         assert captured["compile"]["raw"] == str(orig)          # reel stitched from the
original
168         assert (out / "GX01 - Highlights.mp4").exists()         # named after the original,
not the proxy
169
170
171     def test_resolve_detect_from_validates_pairing(tmp_path):
172         assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
173         with pytest.raises(SystemExit):                         # count mismatch
174             gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
175         with pytest.raises(SystemExit):                         # requires explicit --video
176             gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None,
videos_dir="d"), ["v1"])
177         abs = os.path.join(os.sep, "abs", "p.mp4")
178         assert gh._resolve_detect_from(
179             MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
```

50. assistant (2026-06-17T14:36:34.461Z)

There's no test for the "source missing" error condition. Let me check if there's ANY test that would catch the log format issue - specifically if there's a caplog or similar:

51. user (2026-06-17T14:36:34.993Z)

(Bash completed with no output)

52. assistant (2026-06-17T14:36:36.642Z)

No log capture tests. Now let me check if there's any production code that parses these logs:

53. user (2026-06-17T14:36:37.144Z)

(Bash completed with no output)

54. assistant (2026-06-17T14:36:38.814Z)

No production code parses this log message. Let me now create a test case to see if the condition is actually reachable and functional:

55. user (2026-06-17T14:36:39.324Z)

```
112         logging.getLogger().addHandler(batch_fh)
113
```

```
114         summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
115         logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
116                     len(video_paths), out_dir, segmenter, force)
117
118         for i, src in enumerate(video_paths, 1):
119             name = os.path.basename(src)
120             stem = os.path.splitext(name)[0]
121             reel_name = f"{stem} - Highlights.mp4"
122             dest_reel = os.path.join(out_dir, reel_name)
123             local_reel = os.path.join(local_work_dir, reel_name)
124             # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
125             # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
126             detect_src = detect_from_paths[i - 1] if detect_from_paths else src
127
128             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
129                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
130                 summary["skipped"].append(name)
131                 continue
132             if not os.path.exists(src) or not os.path.exists(detect_src):
133                 logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
134                                i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
135                 summary["failed"].append(name)
136                 continue
137
138             run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
139             run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
140             run_fh.setFormatter(fmt)
141             logging.getLogger().addHandler(run_fh)
```

56. assistant (2026-06-17T14:36:45.584Z)

Perfect! Now I can analyze this clearly. The code on lines 132-136 shows the condition IS reachable and functional. The condition checks:

```python
if not os.path.exists(src) or not os.path.exists(detect_src):
```

This is reachable when:
1. The main video file (`src`) doesn't exist, OR
2. The detection proxy file (`detect_src`) doesn't exist

Now let me verify the logging statement is syntactically correct:

```python
logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s detect_exists=%s)",
               i, len(video_paths), name, os.path.exists(src), os.path.exists(detect_src))
```

This has a format string with 5 `%` placeholders:
1. `%d` for `i`
2. `%d` for `len(video_paths)`
3. `%s` for `name`
4. `%s` for `os.path.exists(src)` (boolean, will be "True" or "False")
5. `%s` for `os.path.exists(detect_src)` (boolean, will be "True" or "False")

And there are exactly 5 arguments passed. This is syntactically correct and will work.

The finding is technically accurate but incomplete:

1. **The change IS real and reachable**: Yes, the condition at line 132 is reachable when either

`src` or `detect_src` doesn't exist.
2. **The log format DOES change**: Yes, it changes from `"[%d/%d] SKIP %s ? source missing"` to include diagnostic info `"(stitch_exists=%s detect_exists=%s)"`.
3. **Does it break anything?** No evidence of breakage because:
   - No tests check the log format of this message
   - No production code parses this warning message
   - The format string is syntactically correct (5 placeholders, 5 args)
   - The message still starts with the same `[%d/%d] SKIP` prefix that users might grep for

SUMMARY: This is a REAL, reachable change. The log output format DOES change, adding diagnostic boolean flags. While this could theoretically break log parsers that depend on the exact format, the codebase has no such parsers. The change is functionally sound but represents a log format modification.

Now let me call the StructuredOutput tool to return the verdict:

57. user (2026-06-17T14:36:47.882Z)

Structured output provided successfully

58. assistant (2026-06-17T14:36:52.161Z)

Summary

The finding is REAL and REACHABLE.

I verified the exact code in the pr-stitch worktree and confirmed:

1. **Location**: File: `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\pr-stitch\tools\generate_highlights.py`, lines 132-134

2. **The actual change**:
   - OLD: `logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)`
   - NEW: `logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s detect_exists=%s)", i, len(video_paths), name, os.path.exists(src), os.path.exists(detect_src))`

3. **Reachability**: The condition `if not os.path.exists(src) or not os.path.exists(detect_src):` is reachable when either the main video or the detection proxy file is missing?a real operational scenario when using the new `--detect-from` feature.

4. **Log format impact**: The message format definitively changes, adding two boolean diagnostic flags `(stitch_exists=True/False detect_exists=True/False)`. This is not a functional regression, but log parsers expecting the exact original format would break.

5. **Mitigation factors**: There is no evidence of log parsers in this codebase that depend on this message format, and tests do not validate it.